

# **Competitive Programming**

## **Section 35: Math Mixed Contest**

Competitive Programming Course

2026-07-10

# Table of contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>How to approach a math mixed contest</b> | <b>3</b>  |
| <b>2</b> | <b>A. Blocked grid paths</b>                | <b>4</b>  |
| <b>3</b> | <b>B. Coprime pairs</b>                     | <b>6</b>  |
| <b>4</b> | <b>C. At least one event</b>                | <b>7</b>  |
| <b>5</b> | <b>Common mistakes</b>                      | <b>8</b>  |
| <b>6</b> | <b>Contest checklist</b>                    | <b>9</b>  |
| <b>7</b> | <b>Exercises</b>                            | <b>10</b> |
|          | 7.1 Paper drills before the judge . . . . . | 10        |
| <b>8</b> | <b>C++ and Python implementation notes</b>  | <b>11</b> |

# 1 How to approach a math mixed contest

Math problems rarely announce their technique. Use statement shape:

| Signal                      | Likely tool                           |
|-----------------------------|---------------------------------------|
| paths with right/down moves | combinations                          |
| avoid one forbidden object  | subtract bad paths                    |
| gcd equals 1                | prime factors and inclusion-exclusion |
| at least one event happens  | complement probability                |
| many values up to $10^6$    | sieve / SPF                           |

## 2 A. Blocked grid paths

Paths from (1,1) to (H,W) use:

```
H-1 down moves  
W-1 right moves
```

so total paths:

```
C(H+W-2, H-1)
```

Paths through blocked cell (R,C):

```
ways start -> blocked  
* ways blocked -> finish
```

Subtract those bad paths.

Worked example:

```
H = 3, W = 4  
blocked = (2,2)
```

All paths need 2 down moves and 3 right moves:

```
C(5,2) = 10
```

Paths from (1,1) to (2,2) need one down and one right:

```
C(2,1) = 2
```

Paths from (2,2) to (3,4) need one down and two right:

```
C(3,1) = 3
```

Bad paths:

```
2 * 3 = 6
```

Good paths:

$$10 - 6 = 4$$

The multiplication is valid because every bad path has a unique prefix ending at the blocked cell and a unique suffix starting from it.

## 3 B. Coprime pairs

Two numbers are not coprime iff they share at least one prime factor.

When processing a new number  $x$ , factor it into distinct primes:

```
x = 2^a * 3^b * 5^c -> distinct primes [2,3,5]
```

For every nonempty subset of these primes, store how many previous numbers were divisible by their product. Inclusion-exclusion gives how many previous numbers share at least one prime factor with  $x$ .

```
previous coprime with x = previous_count - share_any_prime
```

Then add  $x$  to all subset-product counters.

This works because the subset products are square-free: each distinct prime is used at most once.

Example with previous numbers:

```
previous = [6, 10, 21]
x = 30 = 2 * 3 * 5
```

Numbers sharing a factor with 30:

```
divisible by 2: 6,10    -> 2
divisible by 3: 6,21    -> 2
divisible by 5: 10      -> 1
```

The sum  $2+2+1$  counts 6 twice if it has factors 2 and 3, so subtract pair intersections:

```
divisible by 2*3: 6      -> 1
divisible by 2*5: 10     -> 1
divisible by 3*5: none   -> 0
```

Then add triple intersections:

```
divisible by 2*3*5: none -> 0
```

So:

```
share_any = 2 + 2 + 1 - 1 - 1 - 0 + 0 = 3
```

All three previous numbers are not coprime with 30, so there are 0 previous coprime pairs involving this  $x$ .

## 4 C. At least one event

For independent events:

$$P(\text{at least one happens}) = 1 - P(\text{none happen})$$

And:

$$P(\text{none happen}) = \text{product}_i (1 - p_i/q_i)$$

Use modular inverses for each  $q_i$ .

Example:

$$\begin{aligned} P(A) &= 1/2 \\ P(B) &= 1/3 \\ P(C) &= 1/4 \end{aligned}$$

Probability none happen:

$$(1/2) * (2/3) * (3/4) = 1/4$$

Probability at least one happens:

$$1 - 1/4 = 3/4$$

The events may overlap in the outcome space, so adding  $1/2 + 1/3 + 1/4$  would be wrong.

## 5 Common mistakes

Subtracting paths through a blocked cell twice.

Counting prime powers instead of distinct primes in coprime-pair inclusion-exclusion.

Forgetting that value 1 has no prime factors and is coprime with all previous numbers.

Adding probabilities of events directly when they overlap.

## 6 Contest checklist

1. Translate the story into a known counting/probability shape.
2. Identify bad objects if subtraction or complement is easier.
3. Check whether values need preprocessing.
4. Verify edge cases: blocked start/end, value 1, probability 0 or 1.

## 7 Exercises

Treat A as a blocked-grid counting reduction, B as a gcd-based pair count, and C as a complement/inclusion-exclusion event. For each, write the mathematical object being counted before coding.

### 7.1 Paper drills before the judge

1. On a 3 by 4 grid blocked at (2,2), list the two binomial factors for paths through the block and verify the answer is four. Hint: a prefix and suffix are independent only after fixing the block.
2. Process [1,6,10] with an initially empty `seen` table. Write entries for 2,3,5,6,10 after every value. Answer: the new-pair contributions are 0,1,1.
3. For probabilities 1/2 and 1/3, compute both the complement product and the incorrect direct sum. Explain which outcomes direct addition counts twice.

Run the local checker after the paper trace:

```
CP_TARGET=solution python3 sections/35_math_mixed_contest/check.py
```

For B, compare the optimized answer with a nested-loop gcd oracle on random arrays of length at most ten. For A and C, enumerate tiny paths/events before trusting the closed formula. # Model checks before code

For a blocked grid cell, every forbidden path decomposes uniquely into a prefix to that cell and suffix from it, so multiplication is justified. For coprime pairs, factor only distinct primes: using prime powers breaks the inclusion-exclusion events. For probabilities, multiply complements only when the statement grants independence. # Coprime-pair invariant and trace

Let `seen[d]` be the number of earlier values divisible by square-free product `d`. For new `x`, enumerate nonempty subsets of its distinct prime factors. Alternating their `seen[product]` counts gives earlier values sharing a prime; subtract from the processed count to get coprime pairs. Update `seen` only after counting, or `x` pairs with itself. Since  $2*3*5*7*11*13*17 > 10^6$ , a local value has at most seven distinct primes and at most 127 subset products.

Process [1,6,10]: one changes no counter; six increments counters for 2,3,6; ten sees `seen[2]+seen[5]-seen[10]=1`, so it is coprime only with one, then updates 2,5,10. This is the implementation invariant in action.

The blocked-cell subtraction applies to exactly one blocked cell; multiple cells require a different model. The complement product for probabilities requires independent events and denominators nonzero modulo MOD.

## 8 C++ and Python implementation notes

For A, factorial and inverse-factorial arrays need indices through  $H+W$ ; use `long long` for modular products in C++. For B, store SPF and `seen` in dense integer vectors indexed by value, but store the pair answer in `long long`. Generate subset products from distinct factors only. Python uses the same dense lists and should buffer output; do not use a dictionary for every divisor when the one-million value bound permits arrays. For C, normalize `1-none` after subtraction in C++; Python's final `% MOD` already does so.